

E01 — Search in Rotated Sorted Array (Binary Search)

Experiment: 1 | LeetCode: #33 | CO: CO1, CO5 | BT Level: BT4

Problem Statement

Given an integer array `nums` sorted in ascending order that has been **rotated** at some unknown pivot index, and an integer `target`, return the index of `target` in `nums`, or `-1` if it is not present.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [4, 5, 6, 7, 0, 1, 2]`, `target = 0`
Output: `4`

Example 2:

Input: `nums = [4, 5, 6, 7, 0, 1, 2]`, `target = 3`
Output: `-1`

Example 3:

Input: `nums = [1]`, `target = 0`
Output: `-1`

Concept Review: Binary Search on Rotated Array

A rotated sorted array like `[4, 5, 6, 7, 0, 1, 2]` has a property: when you pick a mid-point, **at least one half is always sorted**.

```
[4, 5, 6, 7, 0, 1, 2]
mid = index 3 (value 7)
```

```
Left half  [4, 5, 6, 7] → sorted (nums[lo] ≤ nums[mid])
Right half [0, 1, 2]   → sorted
```

Strategy:

1. Find `mid`.
 2. Determine which half is sorted.
 3. Check if `target` lies in the sorted half → binary search there.
 4. Otherwise search the other half.
-

Algorithm

```

search(nums, target):
    lo = 0, hi = len(nums) - 1

    while lo <= hi:
        mid = lo + (hi - lo) // 2

        if nums[mid] == target: return mid

        // Determine which half is sorted
        if nums[lo] <= nums[mid]: // Left half is sorted
            if nums[lo] <= target < nums[mid]:
                hi = mid - 1 // Target is in left half
            else:
                lo = mid + 1 // Target is in right half
        else: // Right half is sorted
            if nums[mid] < target <= nums[hi]:
                lo = mid + 1 // Target is in right half
            else:
                hi = mid - 1 // Target is in left half

    return -1

```

Implementation

```

def search(nums, target):
    """
    Search for target in rotated sorted array.
    Time: O(log n) | Space: O(1)
    """
    lo, hi = 0, len(nums) - 1

    while lo <= hi:
        mid = lo + (hi - lo) // 2 # Avoid overflow

        if nums[mid] == target:
            return mid

        # Left half is sorted
        if nums[lo] <= nums[mid]:
            if nums[lo] <= target < nums[mid]:
                hi = mid - 1 # Search left
            else:
                lo = mid + 1 # Search right
        # Right half is sorted
        else:
            if nums[mid] < target <= nums[hi]:
                lo = mid + 1 # Search right
            else:
                hi = mid - 1 # Search left

    return -1

```

Detailed Trace

Input: `nums = [4, 5, 6, 7, 0, 1, 2], target = 0`

Iteration	lo	hi	mid	nums[mid]	Action
1	0	6	3	7	<code>nums[lo] = 4 ≤ nums[mid] = 7</code> → left sorted. <code>target = 0</code> not in <code>[4,7)</code> → <code>lo = 4</code>
2	4	6	5	1	<code>nums[lo] = 0 ≤ nums[mid] = 1</code> → left sorted. <code>target = 0</code> in <code>[0,1)</code> → <code>hi = 4</code>
3	4	4	4	0	<code>nums[mid] = 0 == target</code> → return 4 ✓

Input: `nums = [4, 5, 6, 7, 0, 1, 2], target = 3`

Iteration	lo	hi	mid	nums[mid]	Action
1	0	6	3	7	left sorted <code>[4,7]</code> . <code>target = 3</code> not in <code>[4,7)</code> → <code>lo = 4</code>
2	4	6	5	1	left sorted <code>[0,1]</code> . <code>target = 3</code> not in <code>[0,1)</code> → <code>hi = 4</code>
3	4	4	4	0	right sorted <code>[0,2]</code> . <code>target = 3</code> not in <code>(0,2]</code> → <code>hi = 3</code>
—	<code>lo = 4 > hi = 3</code>	loop ends	return -1 ✓		

Edge Cases

```
# Edge case: no rotation
nums = [1, 2, 3, 4, 5]
# Standard binary search – left half always "sorted" → works correctly

# Edge case: single element
nums = [1], target = 1    # → 0
nums = [1], target = 0    # → -1

# Edge case: two elements
nums = [3, 1], target = 1    # → 1
nums = [3, 1], target = 3    # → 0
```

Test Suite

```
def test_search():
    test_cases = [
        ([4, 5, 6, 7, 0, 1, 2], 0, 4),
        ([4, 5, 6, 7, 0, 1, 2], 3, -1),
        ([1], 0, -1),
        ([1], 1, 0),
        ([3, 1], 1, 1),
```

```
        ([5, 1, 3], 5, 0),
        ([1, 3], 3, 1),
        ([2, 3, 4, 5, 6, 7, 8, 1], 6, 4),
    ]
    for nums, target, expected in test_cases:
        result = search(nums, target)
        status = "PASS" if result == expected else "FAIL"
        print(f"{status}: search({nums}, {target}) = {result} (expected {expected})")

test_search()
```

Complexity Analysis

Metric	Value
Time	$O(\log n)$ — search space halved each iteration
Space	$O(1)$ — no extra memory used

Generalization: Find Rotation Index

Finding the index of the minimum element (rotation point):

```
def find_min_index(nums):
    """Find index of minimum element (rotation index).  $O(\log n)$ """
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] > nums[hi]:
            lo = mid + 1    # Min is in right half
        else:
            hi = mid        # Min is in left half (including mid)
    return lo
```

Key Takeaways

- Binary search applies to rotated sorted arrays by identifying the **sorted half**.
 - At each step, one of the two halves is always sorted — use this invariant.
 - The key condition: `nums[lo] <= nums[mid]` determines which half is sorted.
 - Time: $O(\log n)$ — mandatory for this problem.
-

References

- LeetCode #33 — Search in Rotated Sorted Array
- Cormen et al., *Introduction to Algorithms*, Ch. 2.1 (Binary Search)